Experiment 'p_4_2_b_count_even_odd_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume :` 0
Responses containing `expect :` 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_seq
Natural language statement: null
Method signature: p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

Ensures

- `even_count == number_even(inputs)`
- `odd_count == number_odd(inputs)`

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

The method should respect the following contract:

ensures even_count == number_even(inputs), ensures odd_count == number_odd(inputs)

The contract uses the following dafny code:

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLIXJSgSQnOJueilQg4XOyxHZLhp
Model: gpt-5.5-2026-04-23
Created at: 1780652313
Finish reason: stop
Usage: 554 tokens in, and 1184 tokens out

```
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
  } else {
```

```
    var tail_even, tail_odd := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := tail_even + 1;
      odd_count := tail_odd;
    } else {
      assert inputs[0] % 2 == 1;
      even_count := tail_even;
      odd_count := tail_odd + 1;
    }
  }
}
```

Final Program

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count:
    ↪ int)
       ensures even_count == number_even(inputs)
       ensures odd_count == number_odd(inputs)
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
  } else {
    var tail_even, tail_odd := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := tail_even + 1;
      odd_count := tail_odd;
    } else {
      assert inputs[0] % 2 == 1;
      even_count := tail_even;
      odd_count := tail_odd + 1;
    }
  }
}
```

Total Token Usage

Input tokens: 554
Output tokens: 1184
Reasoning tokens: 1024
Sum of 'total tokens': 1738

Experiment Timings

Overall Experiment started at 1780652313150, ended at 1780652358602, lasting 45452ms (45.45 seconds)
Iteration #1 started at 1780652313185, ended at 1780652358601, lasting 45416ms (45.42 seconds)